TP Machine Learning : détecter une cardiomyopathie sur échographies

Niveau moyen · durée estimée : 2 h · Python, scikit-learn, scikit-image

Objectif. Construire de bout en bout un détecteur de cœur anormal à partir d'images : explorer les images, extraire des caractéristiques, comparer des modèles, régler l'un d'eux, évaluer avec les mesures médicales (sensibilité, spécificité, AUC), fixer un seuil pour une sensibilité cible et regarder où le modèle « regarde ».

Données. 600 images 64 × 64 en niveaux de gris, **simulées** par le fichier fourni `echo_gen.py` : une cavité sombre (le sang du ventricule gauche) entourée d'un muscle plus clair (le myocarde), avec le bruit granuleux typique des ultrasons (*speckle*). Classe 1 = cœur pathologique (cavité dilatée, paroi plus fine), classe `0` = normal. Les images sont **synthétiques** : elles servent à apprendre la méthode, pas à diagnostiquer.

Prérequis. `pip install scikit-learn scikit-image matplotlib scipy numpy`. Notions : validation croisée, F1, sensibilité / spécificité, pipeline.

Partie 1 · Exploration des images (10 min)

- Génère le jeu avec `make_dataset(n=600, prevalence=0.3, seed=0)` (importé depuis `echo_gen`).
- Affiche la forme du tableau, l'intervalle des valeurs de pixels et la répartition des classes.
- Affiche 4 images de chaque classe et l'image moyenne de chaque classe. Qu'est-ce qui différencie les deux classes à l'œil ? Pourquoi l'accuracy seule serait-elle trompeuse ?

Partie 2 · Découpage (5 min)

- Sépare 80 % / 20 % de façon stratifiée (`random_state=42`).
- Vérifie que le pourcentage de cas pathologiques est proche dans le train et le test.

Partie 3 · Caractéristiques et comparaison (25 min)

- Écris une fonction qui transforme chaque image en vecteur **HOG** (`skimage.feature.hog`, cellules 16 × 16, blocs 2 × 2).
- Mets-la dans un `Pipeline` via `FunctionTransformer`, suivie d'un `StandardScaler` puis du modèle.
- Compare régression logistique, SVM calibré (`CalibratedClassifierCV(SVC(), ensemble=False)`) et forêt aléatoire par validation croisée à 5 plis (F1).
- Pourquoi HOG convient-il à des images d'échographie ? Pourquoi faire l'extraction dans le pipeline ?

Partie 4 · Optimisation (15 min)

- Avec `GridSearchCV`, règle le SVM : C dans [0,1 ; 1 ; 10 ; 100] et gamma dans [« scale » ; 0,0005 ; 0,005].
- Indice : les paramètres s'appellent `m__estimator__C` et `m__estimator__gamma`. Pourquoi ?
- Le réglage améliore-t-il beaucoup le score ? Qu'en conclus-tu ?

Partie 5 · Évaluation finale (15 min)

- Sur le test, calcule la matrice de confusion, la sensibilité, la spécificité, la précision, le F1 et l'AUC.
- Ici la classe positive est **pathologique** (1). Lis donc chaque case avec cette convention.
- L'incertitude : sur 32 malades, que vaut un seul patient de plus ou de moins ?

Partie 6 · Seuil pour une sensibilité cible (20 min)

- Objectif clinique : détecter au moins 95 % des cœurs pathologiques.
- Obtiens des probabilités honnêtes sur le **train** avec `cross_val_predict(..., method='predict_proba')`, puis cherche le plus grand seuil qui garantit un rappel ≥ 0,95 (`precision_recall_curve`).
- Applique ce seuil au test. Pourquoi ne faut-il jamais choisir le seuil sur le test ? Le résultat est-il meilleur qu'avec 0,5 ? Pourquoi ?

Partie 7 · Où regarde le modèle ? (20 min)

- Choisis une image pathologique du test dont la probabilité est faible mais supérieure à 0,5.
- Méthode d'**occlusion** : masque un carré 12 × 12 (remplacé par la moyenne de l'image), relève la baisse de probabilité, déplace le carré par pas de 4 pixels et affiche la carte obtenue.
- Quelles zones comptent le plus ? Est-ce cohérent avec la médecine ?

> **Bonus.** (1) Ajoute de l'augmentation de données (retournement, petite rotation) et regarde l'effet. (2) Compare HOG à des pixels bruts. (3) Pour de vraies images, le découpage doit se faire **par patient** : explique pourquoi.

Corrigé

Code complet dans `tp_coeur_solution.py` (avec `echo_gen.py`). Résultats d'une exécution réelle (scikit-learn 1.9).

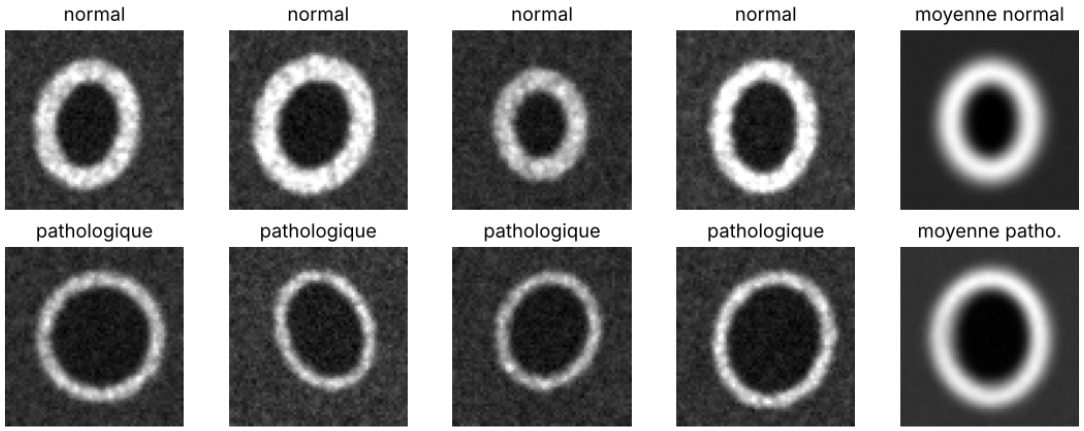
Imports

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from skimage.feature import hog
from sklearn.model_selection import (train_test_split, cross_val_score,
                                     cross_val_predict, StratifiedKFold, GridSearchCV)
from sklearn.preprocessing import StandardScaler, FunctionTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import (confusion_matrix, precision_score, recall_score,
                             f1_score, roc_auc_score, precision_recall_curve)
from echo_gen import make_dataset
```

Partie 1 · Exploration

```
X, y = make_dataset(n=600, prevalence=0.3, seed=0)   # 1 = pathologique, 0 = normal
print("Images :", X.shape, "| valeurs entre",
      round(float(X.min()), 2), "et", round(float(X.max()), 2))
print("Répartition :", {0: int((y == 0).sum()), 1: int((y == 1).sum())},
      "(0 = normal, 1 = pathologique)")
fig, ax = plt.subplots(2, 5, figsize=(10, 4))
for i in range(4):
    ax[0, i].imshow(X[y == 0][i], cmap="gray"); ax[0, i].set_title("normal")
    ax[1, i].imshow(X[y == 1][i], cmap="gray"); ax[1, i].set_title("pathologique")
ax[0, 4].imshow(X[y == 0].mean(0), cmap="gray"); ax[0, 4].set_title("moyenne normal")
ax[1, 4].imshow(X[y == 1].mean(0), cmap="gray"); ax[1, 4].set_title("moyenne patho.")
for a in ax.ravel(): a.axis("off")
plt.tight_layout(); plt.savefig("fig_exploration.png", dpi=110); plt.close()
```

```
Images : (600, 64, 64) | valeurs entre 0.0 et 1.0
Répartition : {0: 440, 1: 160} (0 = normal, 1 = pathologique)
```



600 images 64 × 64, valeurs entre 0 et 1, 440 normales pour 160 pathologiques (27 %). Les images moyennes montrent la différence : cavité plus grande et paroi plus fine pour les cœurs pathologiques. Les classes sont inégales : on suivra le F1, la sensibilité et l'AUC plutôt que l'accuracy.

Partie 2 · Découpage

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
print("Train/Test :", X_tr.shape, X_te.shape,
      "| % patho train =", round(100*y_tr.mean(), 1))
```

```
Train/Test : (480, 64, 64) (120, 64, 64) | % patho train = 26.7
```

Découpage stratifié : 480 images d'entraînement (26,7 % de pathologiques) et 120 de test. Les proportions sont proches, l'évaluation sera comparable.

Partie 3 · Caractéristiques et comparaison

```
def extraire_hog(images):
    return np.stack([hog(im, pixels_per_cell=(16, 16), cells_per_block=(2, 2))
                     for im in images])

def pipe(modele, scaler=True):
    etapes = [("hog", FunctionTransformer(extraire_hog))]
    if scaler: etapes.append(("sc", StandardScaler()))
    return Pipeline(etapes + [("m", modele)])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
models = {
    "Logistique": pipe(LogisticRegression(max_iter=2000)),
    "SVM": pipe(CalibratedClassifierCV(SVC(), ensemble=False)),
    "Random Forest": pipe(RandomForestClassifier(200, random_state=42), scaler=False),
}
for name, m in models.items():
    s = cross_val_score(m, X_tr, y_tr, cv=cv, scoring="f1")
    print(f"{name:14s} F1 CV = {s.mean():.3f} (+/- {s.std():.3f})")
```

```
Logistique     F1 CV = 0.889 (+/- 0.038)
SVM            F1 CV = 0.908 (+/- 0.042)
Random Forest  F1 CV = 0.867 (+/- 0.052)
```

HOG décrit les **contours** (direction et force des bords) : c'est ce qui change entre un cœur normal et dilaté, et c'est assez robuste au bruit granuleux. Dans le pipeline, la validation croisée travaille directement sur les images brutes et le scaler n'est ajusté que sur les plis d'entraînement. Le SVM gagne (F1 ≈ 0,908) devant la logistique (0,889) et la forêt (0,867).

Partie 4 · Optimisation

```
grille = {"m__estimator__C": [0.1, 1, 10, 100],
          "m__estimator__gamma": ["scale", 0.0005, 0.005]}
grid = GridSearchCV(models["SVM"], grille, cv=cv, scoring="f1")
grid.fit(X_tr, y_tr)
print("Meilleurs réglages :", grid.best_params_, "| F1 CV =", round(grid.best_score_, 3))
```

```
Meilleurs réglages : {'m__estimator__C': 1, 'm__estimator__gamma': 0.005} | F1 CV = 0.91
```

Meilleur réglage : C = 1, gamma = 0,005, F1 ≈ 0,91, quasiment comme avant (0,908). Le réglage apporte ici très peu : le problème est limité par le bruit des images, pas par le choix de C. Les noms `m__estimator__C` viennent de l'emboîtement : étape « m » → calibrateur → SVC (« estimator »).

Partie 5 · Évaluation finale

```
best = grid.best_estimator_
pred = best.predict(X_te)
proba = best.predict_proba(X_te)[:, 1]
tn, fp, fn, tp = confusion_matrix(y_te, pred).ravel()
print(f"Matrice : TN={tn} FP={fp} FN={fn} TP={tp}")
print(f"Sensibilité={recall_score(y_te, pred):.3f} Spécificité={tn/(tn+fp):.3f}",
      f"Précision={precision_score(y_te, pred):.3f} F1={f1_score(y_te, pred):.3f}",
      f"AUC={roc_auc_score(y_te, proba):.3f}")
```

```
Matrice : TN=86 FP=2 FN=1 TP=31
Sensibilité=0.969 Spécificité=0.977 Précision=0.939 F1=0.954 AUC=0.998
```

Test : 86 sains bien écartés, 2 fausses alertes, 1 malade manqué, 31 détectés. Sensibilité 0,969, spécificité 0,977, F1 0,954, AUC 0,998. Attention : avec seulement 32 malades dans le test, l'incertitude est d'environ ± 6 points sur la sensibilité. L'AUC proche de 1 vient du fait que les images sont simulées et bien séparées.

Partie 6 · Seuil pour une sensibilité cible

```
# Le seuil est choisi sur le TRAIN (probabilités issues de la validation
# croisée), jamais sur le test.
p_cv = cross_val_predict(best, X_tr, y_tr, cv=cv, method="predict_proba")[:, 1]
prec, rap, seuils = precision_recall_curve(y_tr, p_cv)
seuil = max(s for s, r in zip(seuils, rap[:-1]) if r >= 0.95)
pm = (proba >= seuil).astype(int)
tn2, fp2, fn2, tp2 = confusion_matrix(y_te, pm).ravel()
print(f"Seuil choisi = {seuil:.2f} (au lieu de 0.50)")
print(f"Test : sensibilité={recall_score(y_te, pm):.3f} spécificité={tn2/(tn2+fp2):.3f}",
      f"précision={precision_score(y_te, pm):.3f} | FN={fn2} FP={fp2}")
```

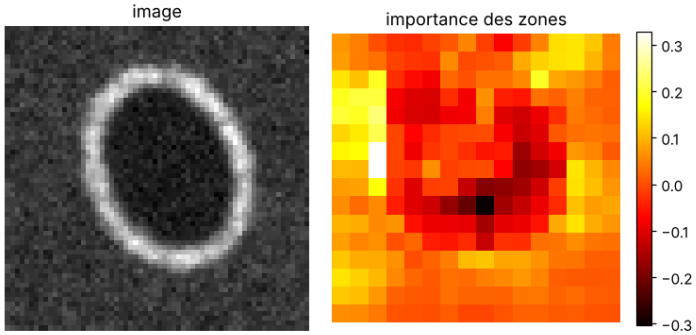
```
Seuil choisi = 0.21 (au lieu de 0.50)
Test : sensibilité=0.969 spécificité=0.966 précision=0.912 | FN=1 FP=3
```

Le seuil 0,21 est choisi sur le train : il garantit un rappel ≥ 95 % en validation croisée. Sur le test, la sensibilité reste 0,969 (1 manqué) mais on gagne 1 fausse alerte de plus (3 au lieu de 2). Leçon : sur un si petit test, un seul patient change tout ; le choix du seuil se justifie par le train, pas par ce que dit le test.

Partie 7 · Où regarde le modèle ?

```
# on prend la patho. la moins "évidente" (proba la plus basse au-dessus
# de 0.5) : la carte est alors plus parlante
cand = np.where((y_te == 1) & (proba > 0.5))[0]
idx = int(cand[np.argmin(proba[cand])])
img = X_te[idx]
base = best.predict_proba(img[None])[0, 1]
carte = np.zeros((16, 16))
for i in range(16):
    for j in range(16):
        im2 = img.copy()
        im2[i*4:i*4+12, j*4:j*4+12] = img.mean()     # on masque un carré de 12x12 pixels
        carte[i, j] = base - best.predict_proba(im2[None])[0, 1]
print(f"Image test n°{idx} : proba patho = {base:.2f} ;",
      f"chute max en masquant une zone = {carte.max():.2f}")
fig, ax = plt.subplots(1, 2, figsize=(7, 3.4))
ax[0].imshow(img, cmap="gray"); ax[0].set_title("image"); ax[0].axis("off")
h = ax[1].imshow(carte, cmap="hot")
ax[1].set_title("importance des zones"); ax[1].axis("off")
plt.colorbar(h, ax=ax[1], fraction=0.046)
plt.tight_layout(); plt.savefig("fig_occlusion.png", dpi=110); plt.close()
```

```
Image test n°100 : proba patho = 0.62 ; chute max en masquant une zone = 0.33
```



On masque un carré à la fois ; plus la probabilité chute, plus la zone compte. Les zones claires de la carte se situent sur le contour de la paroi (bord de la cavité), pas dans le fond de l'image, ce qui est cohérent : c'est la **taille de la cavité et l'épaisseur de la paroi** qui distinguent les classes. Piste : une carte qui s'allumerait sur un coin de l'image signalerait un raccourci à corriger.

> **Pour aller plus loin.** Sur de vraies échographies, on utiliserait un réseau convolutif (CNN) appris de bout en bout, avec un découpage **par patient** pour éviter qu'un même patient se retrouve à la fois dans le train et le test. Ce TP reste un exercice pédagogique sur données simulées.